Give Your AI Searchable Context: RAG on Postgres with pgvector

Mode 2 Phase 1 (Building Blocks), Chapter 3/6 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)
Source: agentfactory.panaversity.org/docs/postgres-ai-crash-course

EN: Build RAG (Retrieval-Augmented Generation) on Postgres + pgvector. For most applications, Postgres IS the AI database -- no separate vector DB needed. Core idea: right information at the right time, everything else stays out.
Roman Urdu: RAG (Retrieval-Augmented Generation) Postgres + pgvector par banao. Zyada tar applications ke liye, Postgres hi AI database hai -- alag vector DB zaroori nahi. Core idea: sahi information sahi waqt par, baaki bahar.

1. Vectors + Extensions + Neon

- A vector = a list of numbers; similar-meaning texts land close together. pgvector (always, pre-installed on Neon) adds the vector type + distance operators + HNSW/IVFFlat indexes.
- One-database payoff: similarity search + a WHERE filter in ONE query, one source of truth.

2. Schema, Chunking, Semantic Search, RAG

- Vectors live in a companion table beside the data -- one source of truth. Chunking SETS THE RECALL CEILING -- if the answer is never in one clean chunk, no model can recover it.
- **RAG = 2 stages:** retrieve (Postgres, top-k ~5) + generate (app: system + chunks + question -> LLM). Most 'AI got it wrong' complaints are actually retrieval failures.

3. Making Search Fast + Good

- Below ~100,000 vectors, exact search is usually fast enough -- benchmark before adding an index. HNSW is Neon's default (~100k-10M, RAM-bound).
- **Eval-driven development:** write a dozen real questions FIRST, re-run on every change. Trace a bad answer: retrieval -> context -> generation -- it's almost always retrieval.
- Filtered search: just add a WHERE clause (metadata/time/permissions). Hybrid search: keyword+vector merged by Reciprocal Rank Fusion (RRF) -- confirm on your eval set, it's not a default win.

4. Multi-Tenancy + Shipping

- 3 isolation levels (weakest to strongest): shared-table+tenant_id, schema-per-tenant, database-per-tenant. Row-Level Security (RLS) enforces the boundary -- but a superuser/owner role bypasses RLS, so test from a non-owner role.
- Ship retrieval as an MCP tool (search_quotes/answer_question) -- a separate, hosted RAG MCP server, distinct from Neon's own dev-time admin MCP.

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
Chunking	Splitting documents into pieces -- sets the hard ceiling on recall quality.	Documents ko pieces mein todna -- recall quality ki hard ceiling set karta.
RAG (retrieve then generate)	Retrieve relevant chunks from Postgres, then generate an answer using them.	Postgres se relevant chunks retrieve karo, phir unse answer generate karo.
HNSW index	Neon's default vector index, good for ~100k-10M vectors, RAM-bound.	Neon ka default vector index, ~100k-10M vectors ke liye achha, RAM-bound.
Reciprocal Rank Fusion (RRF)	A position-based way to merge keyword and vector search results.	Position-based tareeqa jo keyword aur vector search results merge karta.
Row-Level Security (RLS)	Database-enforced tenant isolation -- bypassed by superuser/owner roles, so test from a non-owner role.	Database-enforced tenant isolation -- superuser/owner role bypass karta, isliye non-owner role se test karo.

Quick Self-Check

Question	Answer
Why does chunking set a 'recall ceiling'?	If the answer to a question is never contained in a single clean chunk, no model downstream can recover it -- the retrieval step already lost it.
When should you add a vector index, and when is exact search fine?	Below ~100,000 vectors exact search is usually fast enough -- add an index only once a benchmark shows it's needed.
Where does a bad RAG answer usually come from?	Almost always the retrieval step (wrong chunks returned), not the LLM's generation step.
Why must RLS be tested from a non-owner database role?	A superuser/owner role bypasses Row-Level Security entirely -- testing as owner would hide a real tenant-isolation failure.

Where This Leads / Sources

Leads to: Eval-Driven Development, Build AI Agents, Building a Digital FTE. Full worked example + required tuning exercises in Part 5.

Source: docs/postgres-ai-crash-course/ (this repo), content originally via Zia Tutor AI connector.